Episodio 2 de 14: La carrera entre leer y liberar

Serie «Leer sin que te quiten el libro»: la réplica del paper «A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting» (Euro-Par 2024), explicada para principiantes.

- **Episodio anterior:** episodio 1, «Qué hicimos y qué salió».
- **Episodio siguiente:** episodio 3, «Los rivales: candados, cuentas dobles y basura retrasada».

Lo esencial de este episodio:

- Leer un puntero compartido y apuntarse como usuario son dos pasos. Entre los dos, otro hilo puede liberar el objeto: es la carrera read-reclaim, que acaba en un use-after-free.
- El procesador solo hace una operación indivisible sobre un sitio de la memoria, y el tablón y la libreta están en dos sitios distintos.
- Hay tres familias de solución: poner un candado, retrasar la trituradora o pagar por adelantado en el propio tablón.

Parte 1. El problema, contado con una biblioteca

1.1 Lo que hay que saber antes

Un programa guarda sus datos en la **memoria**. Cada dato vive en una dirección, igual que cada libro vive en un estante numerado. Un **puntero** es un número que dice en qué dirección está algo: es una ficha que pone «estante 42».

Un **hilo** es una tarea que corre en un núcleo del procesador. Tener varios hilos es como tener varias personas trabajando a la vez en el mismo edificio. Pueden ir a la vez y ninguna ve lo que hace la otra hasta que mira.

Cuando un dato ya no hace falta hay que **liberarlo**, es decir, devolver su sitio para que se reutilice. Si no se libera, la memoria se va llenando. Si se libera demasiado pronto, alguien puede ir a buscarlo y encontrarse otra cosa o nada. Eso es el error más temido de esta historia: el **use-after-free**, «usar después de liberar». Según el paper, es una causa habitual de agujeros de seguridad.

1.2 El conteo de referencias

La forma más sencilla de saber cuándo liberar algo es llevar la cuenta de cuánta gente lo está usando.

> Cada libro lleva pegada una libreta con un número: cuántas personas lo usan. Quien empieza a usarlo suma 1. Quien termina resta 1. Quien deja la libreta en 0 sabe que era el último y tira el libro a la trituradora.

Eso es el **conteo de referencias**. En C++ se llama `shared_ptr`, en Rust `Arc`. Con un solo hilo funciona perfecto.

1.3 El tablón: la variable que cambia

Ahora añadimos una pieza clave. En el vestíbulo hay un **tablón** que dice cuál es «el libro del día»: «estante 42». Cualquiera puede leerlo y un bibliotecario puede cambiarlo.

En el paper, ese tablón es un **atomic shared pointer**: una variable compartida, que varios hilos leen y escriben a la vez, y que apunta a un objeto con contador de referencias. Tiene dos operaciones:

- **`load`** (cargar): leer el tablón y quedarte con una referencia al libro.
- **`store`** (guardar): cambiar el tablón para que señale otro libro.

1.4 La carrera, en forma de historia

Marta (un hilo lector) y Beto (un hilo escritor) trabajan a la vez. El libro del estante 42 solo lo «usa» el tablón, así que su libreta marca 1.

1. Marta mira el tablón y lee «estante 42». Se dirige hacia allí.
2. Antes de que llegue, Beto cambia el tablón a «estante 77».
3. Beto va a la libreta del libro 42 y resta 1. Queda en 0. ¡Era el último!
4. Beto tira el libro 42 a la trituradora.
5. Marta llega y quiere sumar 1 en la libreta… de un libro que ya no existe.

Marta está escribiendo sobre una libreta destruida. En el ordenador, ese trozo de memoria puede pertenecer ya a otra cosa. Eso es una **carrera read-reclaim**: una lectura compite con una liberación.

1.5 La misma carrera, en forma de tabla

Momento	Marta (lee)	Beto (escribe)	Tablón	Libreta del libro 42
0			42	1
1	lee «42»		42	1
2		pone «77»	77	1
3		resta 1	77	**0**
4		**tritura el libro**	77	destruida
5	**suma 1: ¡use-after-free!**		77	destruida

1.6 La misma carrera, en forma de código

```
load:                        store(nuevo):
  p = tablon                   viejo = intercambiar(tablon, nuevo)
  // ← AQUÍ puede pasar        viejo.contador -= 1
  //   todo lo de la derecha   si viejo.contador == 0: liberar(viejo)
  p.contador += 1
  devolver p
```

El agujero está entre las dos líneas de `load`. Son dos pasos separados y en medio puede ocurrir cualquier cosa.

1.7 Por qué no se arregla fácil

La idea obvia es hacer los dos pasos «a la vez». El problema es físico: el tablón y la libreta están **en dos sitios distintos de la memoria**, y los procesadores actuales solo pueden hacer una operación indivisible sobre **un sitio**. Es como si solo pudieras tocar una cosa con la mano en cada instante.

Hay tres grandes familias de solución, y todo el paper gira en torno a ellas:

1. **Poner un candado**: mientras alguien lee, nadie cambia nada. Simple, pero se forman colas. *(Mutex y Libstdc++)*
2. **Retrasar la trituradora**: no se destruye nada hasta estar seguros de que nadie va de camino. Rápido, pero la basura se acumula un rato. *(Anderson)*
3. **Pagar por adelantado en el propio tablón**: juntar el puntero y una cuenta en un solo sitio, para que leer y reservar sean el mismo gesto. *(Williams, Folly y los Autores)*

La aportación del paper es una versión de la tercera familia que, además, garantiza que nadie se queda atascado.

Parte 2. Las palabras que hacen falta

2.1 Operaciones atómicas

Una operación es **atómica** si ocurre entera o no ocurre, sin que nadie pueda ver un estado a medias. Las que aparecen aquí son:

Operación	Qué hace	Analogía
`fetch_add` / `fetch_sub`	suma o resta y te dice el valor anterior	arrancar el papelito de un taco de turnos: te llevas tu número y el taco queda actualizado
`exchange`	pone un valor nuevo y te devuelve el viejo	cambiar el cartel y quedarte con el antiguo en la mano
`CAS` (compare-and-swap)	«si vale X, ponlo a Y; si no, no hagas nada y dime cuánto vale»	«solo cambio el cartel si sigue poniendo lo que yo leí»
`DWCAS`	un CAS sobre 16 bytes en vez de 8	lo mismo sobre un cartel doble; existe, pero es más lento

La diferencia entre `fetch_add` y `CAS` es la clave de todo el paper:

- **`fetch_add` nunca falla.** Pides tu turno y lo tienes, aunque haya mil personas.
- **`CAS` puede fallar.** Si alguien cambió el valor justo antes, no pasa nada y tienes que reintentar. Con mucha gente, puedes perder una y otra vez.

2.2 Garantías de progreso: la parte «wait-free» del título

Imagina una puerta por la que tiene que pasar todo el mundo.

Garantía	Qué promete	Analogía	Peligro
Bloqueante	nada, si alguien se detiene	hay una sola llave; quien la tiene se duerme y **todos** esperan	un hilo parado detiene a los demás
Lock-free	alguien siempre avanza	puerta giratoria a empujones: siempre pasa alguien, pero **tú** podrías perder eternamente	un hilo concreto puede quedarse atrás
Wait-free	**todos** acaban en un número máximo de pasos	cada persona tiene su torniquete: pase lo que pase, terminas en N pasos	es lo más difícil de conseguir

Otra forma de verlo: bloqueante es «depende de los demás», lock-free es «depende de la suerte», wait-free es «no depende de nadie».

Wait-free **no significa más rápido**. Significa más predecible. Se puede ser wait-free y lento, igual que se puede ser muy rápido de media y tener a un hilo sufriendo. Esto importará al leer los resultados.

2.3 Caché, líneas y por qué compartir es caro

Este es el concepto que explica casi todas las gráficas.

Cada núcleo tiene una memoria pequeña y rapidísima pegada a él: la **caché**. Los datos se mueven en bloques de 64 bytes llamados **líneas de caché**. Para escribir en una línea, un núcleo debe tenerla **en exclusiva**: hay que quitársela al que la tenía.

> Imagina que todos deben escribir en **un único cuaderno**. No se puede fotocopiar: para escribir hay que tenerlo en la mano. Si 20 personas quieren escribir, el cuaderno viaja de mano en mano. El trabajo real dura un instante; lo que consume tiempo es **pasarse el cuaderno**.

Por eso, cuando todos los hilos escriben en la misma variable, añadir hilos **no acelera nada**: el cuaderno sigue siendo uno. Eso se llama **contención**.

Y aquí entra la distancia. Pasar el cuaderno a tu compañero de mesa es rápido. Enviarlo a otro edificio es lento. En un ordenador con varios **sockets** (varios chips en la misma placa), mandar una línea a otro socket es varias veces más lento que dentro del mismo chip. Por eso, en la máquina del paper, con 4 chips, importa dónde cae cada hilo.

2.4 Cómo leer los números

- **Mop/s**: millones de operaciones por segundo. Más es mejor. Mide **rendimiento**, cuánto trabajo total sale.
- **ns y µs**: nanosegundos y microsegundos (1 µs = 1 000 ns). Menos es mejor. Miden **latencia**, cuánto tarda una operación.
- **Mediana (p50)**: la mitad de las medidas fue más rápida y la otra mitad más lenta. Es la operación típica.

- **p99**: el 99 % fue más rápido que esto. Es «lo malo que puede ponerse». Si la mediana es buena y el p99 horrible, la experiencia es a tirones.

Rendimiento y latencia no son lo mismo. Una autopista puede mover muchos coches por hora aunque cada uno tarde bastante. Por eso el paper mide las dos cosas.

Hay una trampa más. Como lo que viaja es la línea entera, dos datos **distintos** que caen en la misma línea se estorban, aunque nadie los comparta. Se llama **compartición falsa** (*false sharing*), y en esta réplica costó hasta 3,65 veces de rendimiento: ver la Parte 6.6.
